# Software Requirements Specification for Software Engineering: subtitle describing software

Team 22, Ghoti
Hemal Rajput
Gurmanjot Minhas
James Araiche
Lily Porter
Sarah Dorfman

October 19, 2026

# Contents


**1 Reference Material** **iv**
- 1.1 Table of Units . . . iv
- 1.2 Table of Symbols . . . iv
- 1.3 Abbreviations and Acronyms . . . v
- 1.4 Mathematical Notation . . . v

**2 Introduction** **2**
- 2.1 Purpose of Document . . . 2
- 2.2 Scope of Requirements . . . 2
- 2.3 Characteristics of Intended Reader . . . 3
- 2.4 Organization of Document . . . 3

**3 General System Description** **3**
- 3.1 System Context . . . 3
- 3.2 User Characteristics . . . 5
- 3.3 System Constraints . . . 5

**4 Specific System Description** **5**
- 4.0.1 Assumptions . . . 5
- 4.0.2 Theoretical Models . . . 6
- 4.0.3 General Definitions . . . 6
- 4.0.4 Data Definitions . . . 6

**5 Requirements** **6**
- 5.1 Functional Requirements . . . 7
  - 5.1.1 Audio Capture Requirements . . . 7
  - 5.1.2 Session Management Requirements . . . 7
  - 5.1.3 Speech Analysis Requirements . . . 7
  - 5.1.4 Audio Capture Requirements . . . 8
  - 5.1.5 Session Management Requirements . . . 8
  - 5.1.6 Speech Analysis Requirements . . . 9
- 5.2 Nonfunctional Requirements . . . 9
- 5.3 Rationale . . . 10

**6 Likely Changes** **10**

**7 Unlikely Changes** **10**

**8 Traceability Matrices and Graphs** **11**

**9 Development Plan** **14**

**10 Values of Auxiliary Constants** **14**

# Revision History

| Date | Version | Notes |
|---|---|---|
| Date 1 | 1.0 | Notes |
| Date 2 | 1.1 | Notes |

[This template is intended for use by CAS 741. For CAS 741 the template should be used exactly as given, except the Reflection Appendix can be deleted. For the capstone course it is a source of ideas, but shouldn't be followed exactly. The exception is the reflection appendix. All capstone SRS documents should have a reflection appendix. —TPLT]

# 1 Reference Material

This section records information for easy reference.

## 1.1 Table of Units

Throughout this document SI (Système International d'Unités) is employed as the unit system. In addition to the basic units, several derived units are used as described below. For each unit, the symbol is given followed by a description of the unit and the SI name.

| symbol | unit | SI |
|---|---|---|
| m | length | metre |
| kg | mass | kilogram |
| s | time | second |
| °C | temperature | centigrade |
| J | energy | joule |
| W | power | watt ($W = J\,s^{-1}$) |

[Only include the units that your SRS actually uses. —TPLT]

[Derived units, like newtons, pascal, etc, should show their derivation (the units they are derived from) if their constituent units are in the table of units (that is, if the units they are derived from are used in the document). For instance, the derivation of pascals as $Pa = N\,m^{-2}$ is shown if newtons and m are both in the table. The derivations of newtons would not be shown if kg and s are not both in the table. —TPLT]

[The symbol for units named after people use capital letters, but the name of the unit itself uses lower case. For instance, pascals use the symbol Pa, watts use the symbol W, teslas use the symbol T, newtons use the symbol N, etc. The one exception to this is degree Celsius. Details on writing metric units can be found on the NIST web-page. —TPLT]

[The table should include the basic units and derived units that are used, with basic and derived defined as by SI. Units that aren't named, aren't normally in the table of units. For instance, the units $m/s^2$ for acceleration won't normally appear in the table. —TPLT]

## 1.2 Table of Symbols

The table that follows summarizes the symbols used in this document along with their units. The choice of symbols was made to be consistent with the heat transfer literature and with existing documentation for solar water heating systems. The symbols are listed in alphabetical order.

| symbol | unit | description |
|---|---|---|
| $A_C$ | $m^2$ | coil surface area |

| | | |
|---|---|---|
| $A_{in}$ | $m^2$ | surface area over which heat is transferred in |

[Use your problems actual symbols. The si package is a good idea to use for units. —TPLT]

## 1.3 Abbreviations and Acronyms

| symbol | description |
|---|---|
| A | Assumption |
| DD | Data Definition |
| GD | General Definition |
| GS | Goal Statement |
| IM | Instance Model |
| LC | Likely Change |
| PS | Physical System Description |
| R | Requirement |
| SRS | Software Requirements Specification |
| Software Engineering | [put an expanded version of your program name here (as appropriate) —TPLT] |
| TM | Theoretical Model |

[Add any other abbreviations or acronyms that you add —TPLT]

## 1.4 Mathematical Notation

[This section is optional, but should be included for projects that make use of notation to convey mathematical information. For instance, if typographic conventions (like bold face font) are used to distinguish matrices, this should be stated here. If symbols are used to show mathematical operations, these should be summarized here. In some cases the easiest way to summarize the notation is to point to a text or other source that explains the notation. —TPLT]

[This section was added to the template because some students use very domain specific notation. This notation will not be readily understandable to people outside of your domain. It should be explained. —TPLT]

[This SRS template is based on Smith and Lai (2005); Smith et al. (2007); Smith and Koothoor (2016). It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. Although the bits shown below do not include type information, you may need to add this information for your problem. If you are unsure, please can ask the instructor. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. A separate template is provided for this. For program families you should look at Smith (2006); Smith et al. (2017). Single family member programs are often programs based on a single physical model. General purpose tools are usually documented as a family. Families of physical models also come up. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. For writing an SRS, and for reading one for the first time, the suggested order of sections is:

- Goal Statement
- Instance Models
- Requirements
- Introduction
- Specific System Description

—TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level. For instance, there will be overlap between the scope section and the assumptions, but the scope section will not go into as much detail as the assumptions section.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. Other information will be derived, like General Definitions. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone that meets the Characteristics of the Intended Reader (Section 2.3) can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

# 2 Introduction

[The introduction section is written to introduce the problem. It starts general and focuses on the problem domain. The general advice is to start with a paragraph or two that describes the problem, followed by a "roadmap" paragraph. A roadmap orients the reader by telling them what sub-sections to expect in the Introduction section. —TPLT]

## 2.1 Purpose of Document

[This section summarizes the purpose of the SRS document. It does not focus on the problem itself. The problem is described in the "Problem Description" section (Section **??**). The purpose is for the document in the context of the project itself, not in the context of this course. Although the "purpose" of the document is to get a grade, you should not mention this. Instead, "fake it" as if this is a real project. The purpose section will be similar between projects. The purpose of the document is the purpose of the SRS, including communication, planning for the design stage, etc. —TPLT]

## 2.2 Scope of Requirements

[Modelling the real world requires simplification. The full complexity of the actual physics, chemistry, biology is too much for existing models, and for existing computational solution techniques. Rather than say what is in the scope, it is usually easier to say what is not. You can think of it as the scope is initially everything, and then it is constrained to create the actual scope. For instance, the problem can be restricted to 2 dimensions, or it can ignore the effect of temperature (or pressure) on the material properties, etc. —TPLT]

[The scope section is related to the assumptions section (Section **??**). However, the scope and the assumptions are not at the same level of abstraction. The scope is at a high level. The focus is on the "big picture" assumptions. The assumptions section lists, and describes, all of the assumptions. —TPLT]

[The scope section is relevant for later determining typical values of inputs. The scope should make it clear what inputs are reasonable to expect. This is a distinction between scope and context (context is a later section). Scope affects the inputs while context affects how the software will be used. —TPLT]

### 2.3 Characteristics of Intended Reader

[This section summarizes the skills and knowledge of the readers of the SRS. It does NOT have the same purpose as the "User Characteristics" section (Section 3.2). The intended readers are the people that will read, review and maintain the SRS. They are the people that will conceivably design the software that is intended to meet the requirements. The user, on the other hand, is the person that uses the software that is built. They may never read this SRS document. Of course, the same person could be a "user" and an "intended reader." —TPLT]

[The intended reader characteristics should be written as unambiguously and as specifically as possible. Rather than say, the user should have an understanding of physics, say what kind of physics and at what level. For instance, is high school physics adequate, or should the reader have had a graduate course on advanced quantum mechanics? —TPLT]

### 2.4 Organization of Document

[This section provides a roadmap of the SRS document. It will help the reader orient themselves. It will provide direction that will help them select which sections they want to read, and in what order. This section will be similar between project. —TPLT]

[Include a reference to the template (Smith and Lai (2005); Smith et al. (2007); Smith and Koothoor (2016)) you are using in the documentation in the Organization of the Document section. —TPLT]

## 3 General System Description

This section provides general information about the system. It identifies the interfaces between the system and its environment, describes the user characteristics and lists the system constraints. [This text can likely be borrowed verbatim. —TPLT]

[The purpose of this section is to provide general information about the system so the specific requirements in the next section will be easier to understand. The general system description section is designed to be changeable independent of changes to the functional requirements documented in the specific system description. The general system description provides a context for a family of related models. The general description can stay the same, while specific details are changed between family members. —TPLT]

### 3.1 System Context

[Your system context will include a figure that shows the abstract view of the software. Often in a scientific context, the program can be viewed abstractly following the design pattern of Inputs → Calculations → Outputs. The system context will therefore often follow this pattern. The user provides inputs, the system does the calculations, and then provides the outputs to the user. The figure should not show all of the inputs, just an abstract view of the main categories of inputs (like material properties, geometry, etc.). Likewise, the

outputs should be presented from an abstract point of view. In some cases the diagram will show other external entities, besides the user. For instance, when the software product is a library, the user will be another software program, not an actual end user. If there are system constraints that the software must work with external libraries, these libraries can also be shown on the System Context diagram. They should only be named with a specific library name if this is required by the system constraint. —TPLT]




Figure 1: System Context

[For each of the entities in the system context diagram its responsibilities should be listed. Whenever possible the system should check for data quality, but for some cases the user will need to assume that responsibility. The list of responsibilities should be about the inputs and outputs only, and they should be abstract. Details should not be presented here. However, the information should not be so abstract as to just say "inputs" and "outputs". A summarizing phrase can be used to characterize the inputs. For instance, saying "material properties" provides some information, but it stays away from the detail of listing every required properties. —TPLT]

- User Responsibilities:
  - 
- Software Engineering Responsibilities:
  - Detect data type mismatch, such as a string of characters instead of a floating point number
  - 

[Identify in what context the software will typically be used. Is it for exploration? education? engineering work? scientific work?. Identify whether it will be used for mission-critical or safety-critical applications. —TPLT] [This additional context information is needed to determine how much effort should be devoted to the rationale section. If the application is safety-critical, the bar is higher. This is currently less structured, but analogous to, the idea to the Automotive Safety Integrity Levels (ASILs) that McSCert uses in their automotive hazard analyses. —TPLT]

## 3.2 User Characteristics

[This section summarizes the knowledge/skills expected of the user. Measuring usability, which is often a required non-function requirement, requires knowledge of a typical user. As mentioned above, the user is a different role from the "intended reader," as given in Section 2.3. As in Section 2.3, the user characteristics should be specific an unambiguous. For instance, "The end user of Software Engineering should have an understanding of undergraduate Level 1 Calculus and Physics." —TPLT]

## 3.3 System Constraints

[System constraints differ from other type of requirements because they limit the developers' options in the system design and they identify how the eventual system must fit into the world. This is the only place in the SRS where design decisions can be specified. That is, the quality requirement for abstraction is relaxed here. However, system constraints should only be included if they are truly required. —TPLT]

# 4 Specific System Description

### 4.0.1 Assumptions

- **A1: Single-word attempts.** Each attempt consists of a single spoken target word.
- **A2: Speaker population.** Each speaker is a monolingual North American English-speaking child aged 4–6.
- **A3: Expected phoneme sequence.** Each target word is drawn from a standardized word set and has a predefined expected phoneme sequence $T = (t_1, \ldots, t_n)$.
- **A4: Acceptable pronunciation variants.** Each target word has a finite, predefined set of acceptable phoneme variants $V_i$ at each position $i$, where $t_i \in V_i$.
  - These sets are established in consultation with an SLP advisor to account for acceptable pronunciation variation.
- **A5: Substitution errors.** Each substitution error involves replacing exactly one expected phoneme with a different phoneme at the same position.
- **A6: Sequence length mismatch.** Any difference between the recognized phoneme sequence length and the expected length $n$ is assumed to indicate at least one omission or addition.
- **A7: SLP supervision.** Each session is conducted under the direct supervision of an SLP.

- **A8: Unintentional pronunciation errors.** All pronunciation errors made by the speaker are assumed to be unintentional.
- **A9: Recording conditions.** Each recording contains speech from only one speaker and is captured in a quiet environment with minimal background noise.
- **A10: Calibrated phoneme confidence.** The recognizer's per-phoneme posterior probabilities are assumed to be calibrated, such that predicted confidence values correspond to observed frequencies of correct recognition.
- **A11: Session devices.** SLPs are assumed to conduct sessions using an iPad, Android tablet, or computer equipped with a microphone and a modern web browser.

#### 4.0.2 Theoretical Models

- **TM1: Phoneme recognition.** $R(x) = (P, \mathbf{c})$, where $x$ is the recorded audio, $P = (p_1, \ldots, p_m)$ is the recognized phoneme sequence, and $\mathbf{c} = (c_1, \ldots, c_m)$ are per-phoneme posteriors with $c_j \in [0, 1]$.
  - Abstract mapping from audio to phonemes with confidence.
- **TM2: Phoneme sequence comparison.** Given $T$ (expected) and $P$ (recognized) with $|P| = n$, pair $t_i$ with $p_i$ for $i = 1, \ldots, n$.
  - Positional comparison is sufficient for substitutions, which are the only errors in scope.

#### 4.0.3 General Definitions

- **GD1: Substitution at position $i$.** $s_i = 1$ if $p_i \notin V_i$ and $(t_i \to p_i) \in L$, otherwise $s_i = 0$, where $L$ is the predetermined list of substitution errors.

#### 4.0.4 Data Definitions

- **DD1: Word position.** $\text{pos}(i)$ is *beginning* if $i = 1$; *end* if $i = n$; otherwise *middle*.
  - Used to report where in the word each substitution occurs.

## 5 Requirements

[The requirements refine the goal statement. They will make heavy use of references to the instance models. —TPLT]

This section provides the functional requirements, the business tasks that the software is expected to complete, and the nonfunctional requirements, the qualities that the software is expected to exhibit.

## 5.1 Functional Requirements

### 5.1.1 Audio Capture Requirements

- **FR-AC1:** The system shall capture the patient's spoken response as audio for each attempt.
  **Rationale:** Every analysis result is derived from the recording of an attempt.

- **FR-AC2:** The system shall use captured audio only to analyze the attempt it was captured for.
  **Rationale:** The system is scoped to data analysis only. Limiting the use of children's audio reduces privacy exposure under PHIPA.

- **FR-AC3:** The system shall flag to the SLP any recording that fails the usability limits (noise, clipping, duration).
  **Rationale:** Noisy or malformed audio is a main cause of misclassification, so the SLP must know when a result may be unreliable.

### 5.1.2 Session Management Requirements

- **FR-SM1:** The system shall allow the SLP to set up a session by selecting a patient, one or more target phonemes, and target items.
  **Rationale:** Target phonemes and items define what is practised and what the analysis compares against. Selectable phonemes are limited to those covered by the predetermined substitution list (FR-SA1).

- **FR-SM2:** The system shall allow the SLP to start, repeat, skip and end each attempt, and to end the session.
  **Rationale:** The SLP supervises the session and controls its pace.

- **FR-SM3:** The system shall allow the SLP to record, for each attempt, whether the target was produced independently, after a semantic cue, or after repetition.
  **Rationale:** The amount of support a patient needs is a clinical indicator of progress and is tracked across sessions.

- **FR-SM4:** The system shall allow the SLP to override the system's classification of any attempt.
  **Rationale:** The system's output is assistive and not diagnostic, so the SLP's clinical judgement takes precedence.

### 5.1.3 Speech Analysis Requirements

- **FR-SA1:** The system shall identify, from the captured audio of each attempt, substitutions from a predetermined list of common substitution errors, reporting the expected phoneme, the produced phoneme, and the word position.

**Rationale:** Substitutions are the only error type in scope for the proof of concept, and accuracy is only validated for the listed errors. Word position supports the position-based visualization.

- **FR-SA2:** The system shall not identify a substitution when the produced phoneme matches any acceptable variant defined for the target item.
  **Rationale:** Accepted North American English pronunciation varies by dialect, and treating those variants as errors would produce false substitutions.

- **FR-SA3:** The system shall report a confidence value with each result and shall classify an attempt as "unclassifiable" when confidence is below the confidence threshold determined by the validation study.
  **Rationale:** The SLP can weigh uncertain results, and the system avoids forcing a classification on poor input, which reduces over-reliance on the tool.

#### 5.1.4 Audio Capture Requirements

- **FR-AC1:** The system shall capture the patient's spoken response as audio for each attempt.
  **Rationale:** Every analysis result is derived from the recording of an attempt.

- **FR-AC2:** The system shall use captured audio only to analyze the attempt it was captured for.
  **Rationale:** The system is scoped to data analysis only. Limiting the use of children's audio reduces privacy exposure under PHIPA.

- **FR-AC3:** The system shall flag to the SLP any recording that fails the usability limits (noise, clipping, duration).
  **Rationale:** Noisy or malformed audio is a main cause of misclassification, so the SLP must know when a result may be unreliable.

#### 5.1.5 Session Management Requirements

- **FR-SM1:** The system shall allow the SLP to set up a session by selecting a patient, one or more target phonemes, and target items.
  **Rationale:** Target phonemes and items define what is practised and what the analysis compares against. Selectable phonemes are limited to those covered by the predetermined substitution list (FR-SA1).

- **FR-SM2:** The system shall allow the SLP to start, repeat, skip and end each attempt, and to end the session.
  **Rationale:** The SLP supervises the session and controls its pace.

- **FR-SM3:** The system shall allow the SLP to record, for each attempt, whether the target was produced independently, after a semantic cue, or after repetition.

**Rationale:** The amount of support a patient needs is a clinical indicator of progress and is tracked across sessions.

- **FR-SM4:** The system shall allow the SLP to override the system's classification of any attempt.
  **Rationale:** The system's output is assistive and not diagnostic, so the SLP's clinical judgement takes precedence.

#### 5.1.6 Speech Analysis Requirements

- **FR-SA1:** The system shall identify, from the captured audio of each attempt, substitutions from a predetermined list of common substitution errors, reporting the expected phoneme, the produced phoneme, and the word position.
  **Rationale:** Substitutions are the only error type in scope for the proof of concept, and accuracy is only validated for the listed errors. Word position supports the position-based visualization.

- **FR-SA2:** The system shall not identify a substitution when the produced phoneme matches any acceptable variant defined for the target item.
  **Rationale:** Accepted North American English pronunciation varies by dialect, and treating those variants as errors would produce false substitutions.

- **FR-SA3:** The system shall report a confidence value with each result and shall classify an attempt as "unclassifiable" when confidence is below the confidence threshold determined by the validation study.
  **Rationale:** The SLP can weigh uncertain results, and the system avoids forcing a classification on poor input, which reduces over-reliance on the tool.

[Every IM should map to at least one requirement, but not every requirement has to map to a corresponding IM. —TPLT]

## 5.2 Nonfunctional Requirements

[List your nonfunctional requirements. You may consider using a fit criterion to make them verifiable. —TPLT] [The goal is for the nonfunctional requirements to be unambiguous, abstract and verifiable. This isn't easy to show succinctly, so a good strategy may be to give a "high level" view of the requirement, but allow for the details to be covered in the Verification and Validation document. —TPLT] [An absolute requirement on a quality of the system is rarely needed. For instance, an accuracy of 0.0101 % is likely fine, even if the requirement is for 0.01 % accuracy. Therefore, the emphasis will often be more on describing now well the quality is achieved, through experimentation, and possibly theory, rather than meeting some bar that was defined a priori. —TPLT] [You do not need an entry for correctness in your NFRs. The purpose of the SRS is to record the requirements that need to be satisfied for correctness. Any statement of correctness would just be redundant.

Rather than discuss correctness, you can characterize how far away from the correct (true) solution you are allowed to be. This is discussed under accuracy. —TPLT]

NFR1: **Accuracy** [Characterize the accuracy by giving the context/use for the software. Maybe something like, "The accuracy of the computed solutions should meet the level needed for <engineering or scientific application>. The level of accuracy achieved by Software Engineering shall be described following the procedure given in Section X of the Verification and Validation Plan." A link to the VnV plan would be a nice extra. —TPLT]

NFR2: **Usability** [Characterize the usability by giving the context/use for the software. You should likely reference the user characteristics section. The level of usability achieved by the software shall be described following the procedure given in Section X of the Verification and Validation Plan. A link to the VnV plan would be a nice extra. —TPLT]

NFR3: **Maintainability** [The effort required to make any of the likely changes listed for Software Engineering should be less than FRACTION of the original development time. FRACTION is then a symbolic constant that can be defined at the end of the report. —TPLT]

NFR4: **Portability** [This NFR is easier to write than the others. The systems that Software Engineering should run on should be listed here. When possible the specific versions of the potential operating environments should be given. To make the NFR verifiable a statement could be made that the tests from a given section of the VnV plan can be successfully run on all of the possible operating environments. —TPLT]

- Other NFRs that might be discussed include verifiability, understandability and reusability.

### 5.3 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, modelling decisions, assumptions and typical values. —TPLT]

## 6 Likely Changes

LC1: [Give the likely changes, with a reference to the related assumption (aref), as appropriate. —TPLT]

## 7 Unlikely Changes

LC2: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

# 8 Traceability Matrices and Graphs

The purpose of the traceability matrices is to provide easy references on what has to be additionally modified if a certain component is changed. Every time a component is changed, the items in the column of that component that are marked with an "X" may have to be modified as well. Table 1 shows the dependencies of theoretical models, general definitions, data definitions, and instance models with each other. Table 2 shows the dependencies of instance models, requirements, and data constraints on each other. Table 3 shows the dependencies of theoretical models, general definitions, data definitions, instance models, and likely changes on the assumptions.

[You will have to modify these tables for your problem. —TPLT]

[The traceability matrix is not generally symmetric. If GD1 uses A1, that means that GD1's derivation or presentation requires invocation of A1. A1 does not use GD1. A1 is "used by" GD1. —TPLT]

[The traceability matrix is challenging to maintain manually. Please do your best. In the future tools (like Drasil) will make this much easier. —TPLT]

| | TM?? | TM?? | TM?? | GD?? | GD?? | DD?? | DD?? | DD?? | DD?? | IM?? | IM?? | IM? |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| TM?? | | | | | | | | | | | | |
| TM?? | | | X | | | | | | | | | |
| TM?? | | | | | | | | | | | | |
| GD?? | | | | | | | | | | | | |
| GD?? | X | | | | | | | | | | | |
| DD?? | | | | X | | | | | | | | |
| DD?? | | | | X | | | | | | | | |
| DD?? | | | | | | | | | | | | |
| DD?? | | | | | | | | X | | | | |
| IM?? | | | | | X | X | X | | | | X | |
| IM?? | | | | | X | | X | | X | X | | |
| IM?? | | X | | | | | | | | | | |
| IM?? | | X | X | | | | X | X | X | | X | |

Table 1: Traceability Matrix Showing the Connections Between Items of Different Sections

The purpose of the traceability graphs is also to provide easy references on what has to be additionally modified if a certain component is changed. The arrows in the graphs represent dependencies. The component at the tail of an arrow is depended on by the component at the head of that arrow. Therefore, if a component is changed, the components that it points to should also be changed. Figure **??** shows the dependencies of theoretical models, general definitions, data definitions, instance models, likely changes, and assumptions on each other.

| | IM?? | IM?? | IM?? | IM?? | ?? | R?? | R?? |
|---|---|---|---|---|---|---|---|
| IM?? | | X | | | | X | X |
| IM?? | X | | | X | | X | X |
| IM?? | | | | | | X | X |
| IM?? | | X | | | | X | X |
| R?? | | | | | | | |
| R?? | | | | | | X | |
| R?? | | | | | X | | |
| R?? | X | X | | | | X | X |
| R?? | X | | | | | | |
| R?? | | X | | | | | |
| R?? | | | X | | | | |
| R?? | | | | X | | | |
| R?? | | | X | X | | | |
| R?? | | X | | | | | |
| R?? | | X | | | | | |

Table 2: Traceability Matrix Showing the Connections Between Requirements and Instance Models

| | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? | A?? |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| TM?? | X | | | | | | | | | | | | | | | | | | |
| TM?? | | | | | | | | | | | | | | | | | | | |
| TM?? | | | | | | | | | | | | | | | | | | | |
| GD?? | | X | | | | | | | | | | | | | | | | | |
| GD?? | | | X | X | X | X | | | | | | | | | | | | | |
| DD?? | | | | | | | X | X | X | | | | | | | | | | |
| DD?? | | | X | X | | | | | | X | | | | | | | | | |
| DD?? | | | | | | | | | | | | | | | | | | | |
| DD?? | | | | | | | | | | | | | | | | | | | |
| IM?? | | | | | | | | | | | X | X | | X | X | X | | | X |
| IM?? | | | | | | | | | | | | X | X | | | X | X | X | |
| IM?? | | | | | | | | | | | | | | X | | | | | X |
| IM?? | | | | | | | | | | | | | X | | | | | X | |
| LC?? | | | | X | | | | | | | | | | | | | | | |
| LC?? | | | | | | | | X | | | | | | | | | | | |
| LC?? | | | | | | | | | X | | | | | | | | | | |
| LC?? | | | | | | | | | | | X | | | | | | | | |
| LC?? | | | | | | | | | | | | X | | | | | | | |
| LC?? | | | | | | | | | | | | | | | X | | | | |

Table 3: Traceability Matrix Showing the Connections Between Assumptions and Other Items

Figure ?? shows the dependencies of instance models, requirements, and data constraints on each other.

# 9 Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could should the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

# 10 Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

# References

W. Spencer Smith. Systematic development of requirements documentation for general purpose scientific computing software. In *Proceedings of the 14th IEEE International Requirements Engineering Conference, RE 2006*, pages 209–218, Minneapolis / St. Paul, Minnesota, 2006. URL http://www.ifi.unizh.ch/req/events/RE06/.

W. Spencer Smith and Nirmitha Koothoor. A document-driven method for certifying scientific computing software for use in nuclear safety analysis. *Nuclear Engineering and Technology*, 48(2):404–418, April 2016. ISSN 1738-5733. doi: http://dx.doi.org/10.1016/j.net.2015.11.008. URL http://www.sciencedirect.com/science/article/pii/S1738573315002582.

W. Spencer Smith and Lei Lai. A new requirements template for scientific computing. In J. Ralyté, P. Ågerfalk, and N. Kraiem, editors, *Proceedings of the First International Workshop on Situational Requirements Engineering Processes – Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP'05*, pages 107–121, Paris, France, 2005. In conjunction with 13th IEEE International Requirements Engineering Conference.

W. Spencer Smith, Lei Lai, and Ridha Khedri. Requirements analysis for engineering computation: A systematic approach for improving software reliability. *Reliable Computing, Special Issue on Reliable Engineering Computation*, 13(1):83–107, February 2007.

W. Spencer Smith, John McCutchan, and Jacques Carette. Commonality analysis for a family of material models. Technical Report CAS-17-01-SS, McMaster University, Department of Computing and Software, 2017.

[The following is not part of the template, just some things to consider when filing in the template. —TPLT]
[Grammar, flow and LaTeXadvice:

- For Mac users `*.DS_Store` should be in `.gitignore`
- LaTeX and formatting rules
  - Variables are italic, everything else not, includes subscripts (link to document)
    - Conventions
    - Watch out for implied multiplication
  - Use BibTeX
  - Use cross-referencing
- Grammar and writing rules
  - Acronyms expanded on first usage (not just in table of acronyms)
  - "In order to" should be "to"

—TPLT]
[Advice on using the template:

- Difference between physical and software constraints
- Properties of a correct solution means *additional* properties, not a restating of the requirements (may be "not applicable" for your problem). If you have a table of output constraints, then these are properties of a correct solution.
- Assumptions have to be invoked somewhere
- "Referenced by" implies that there is an explicit reference
- Think of traceability matrix, list of assumption invocations and list of reference by fields as automatically generatable
- If you say the format of the output (plot, table etc), then your requirement could be more abstract

—TPLT]

# Appendix — Reflection

[Not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?